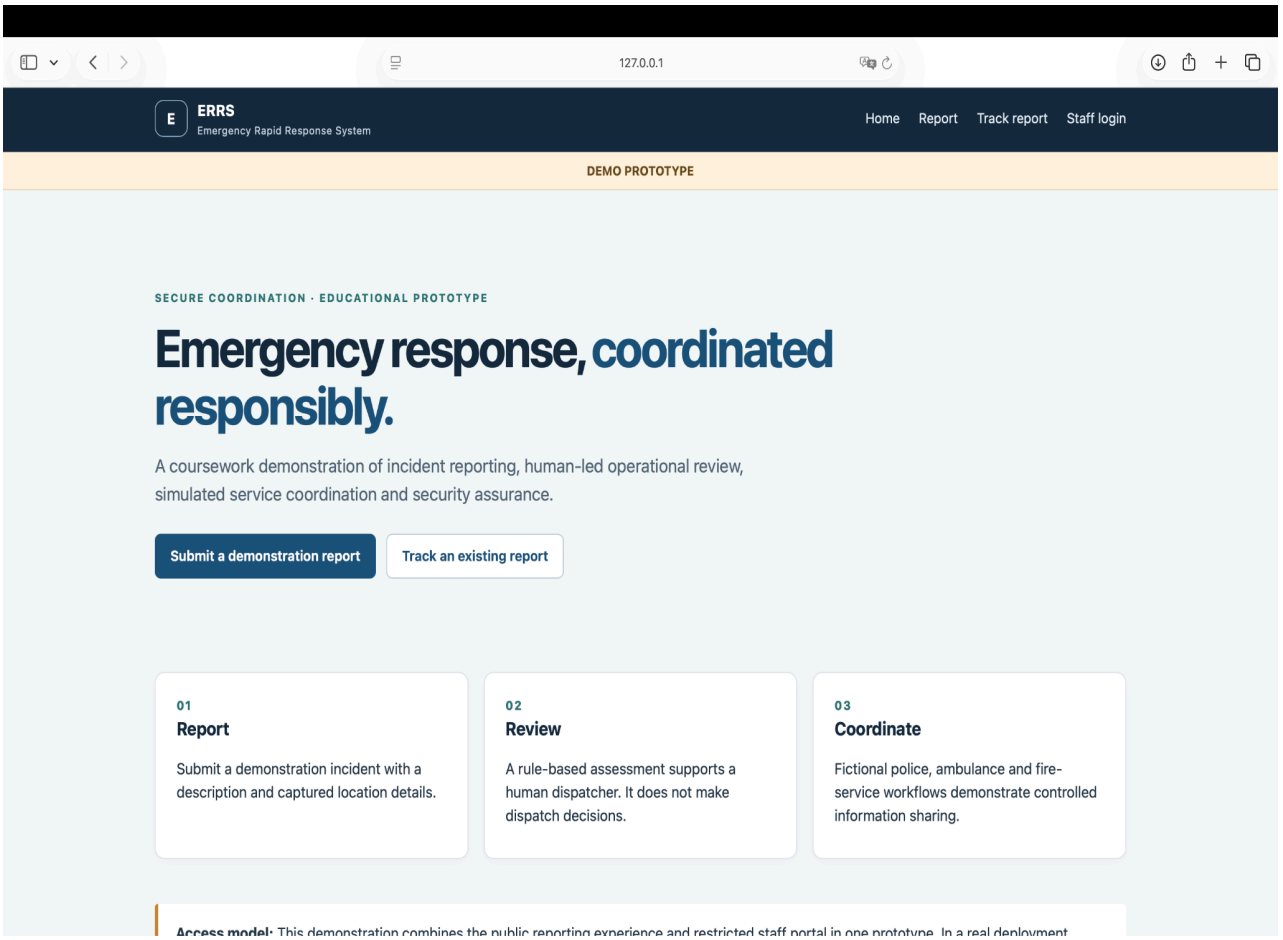
SECURITY ASSURANCE PORTFOLIO

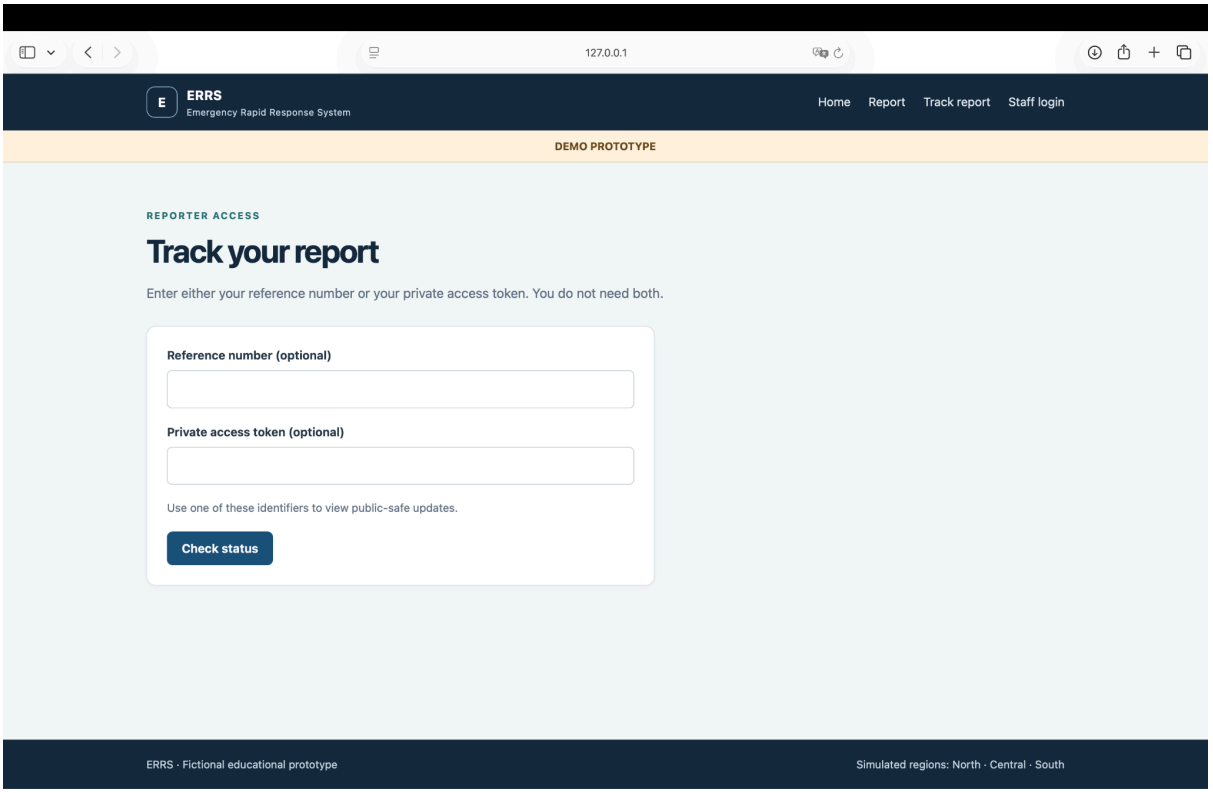
Portfolio Overview

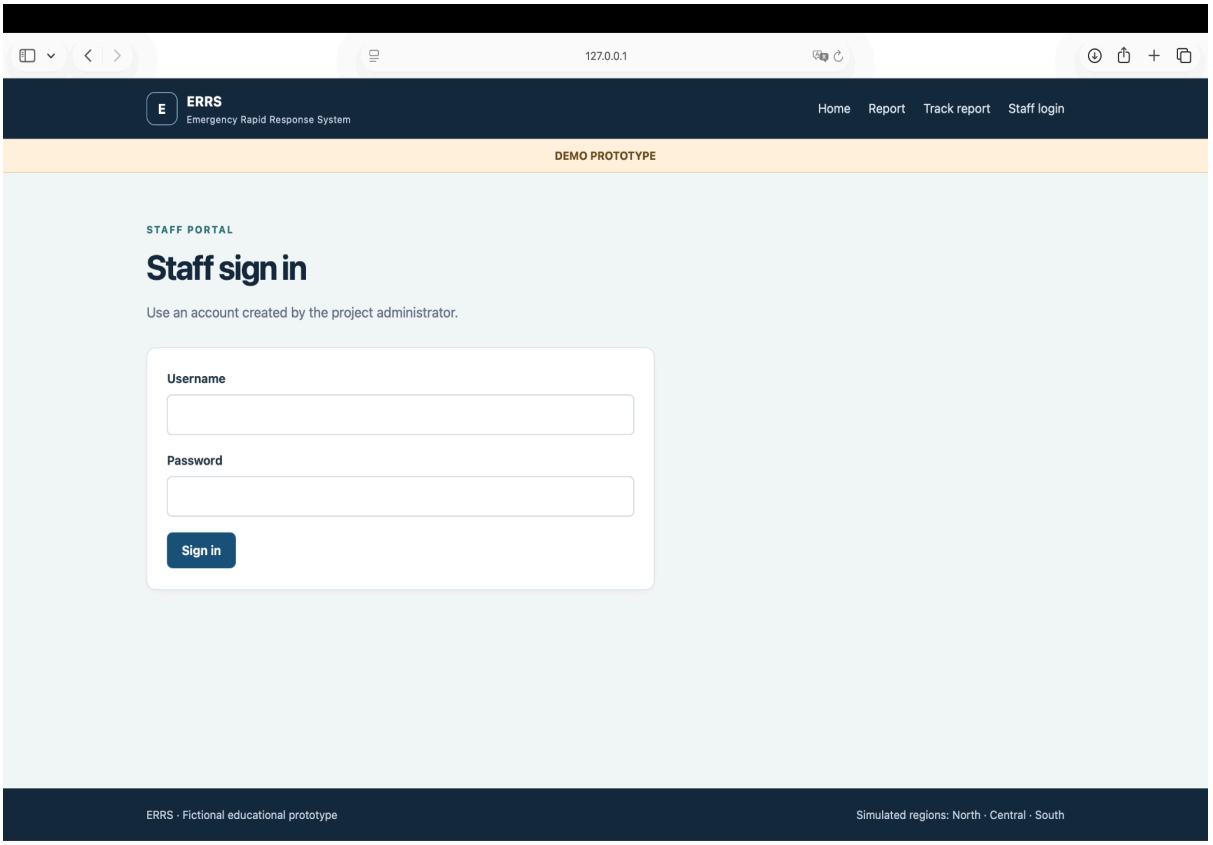
This portfolio contains screenshots and test results of the Emergency Rapid Response System (ERRS). It demonstrates the main application features,security implementation, consensus experiments and testing/results. It provides real evidence to support the main coursework report and shows how the system works in practice.

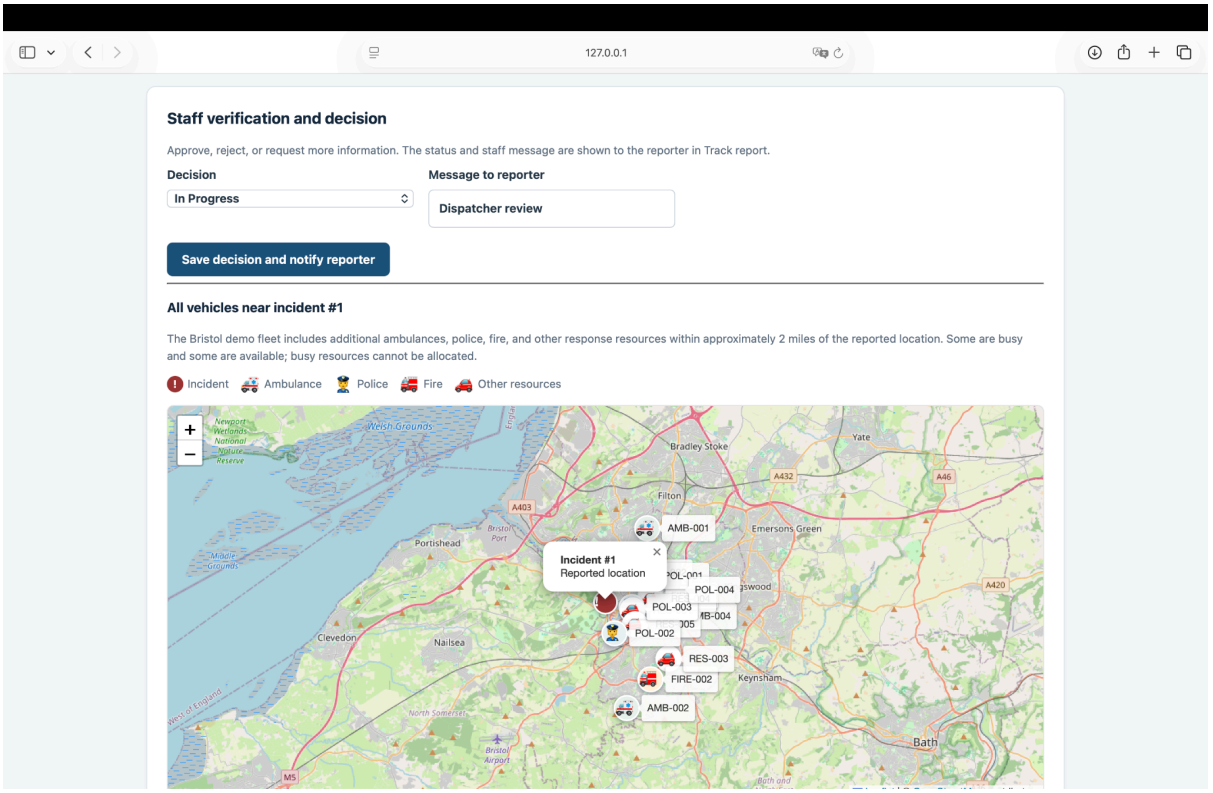
Figure 1 - Application Evidence

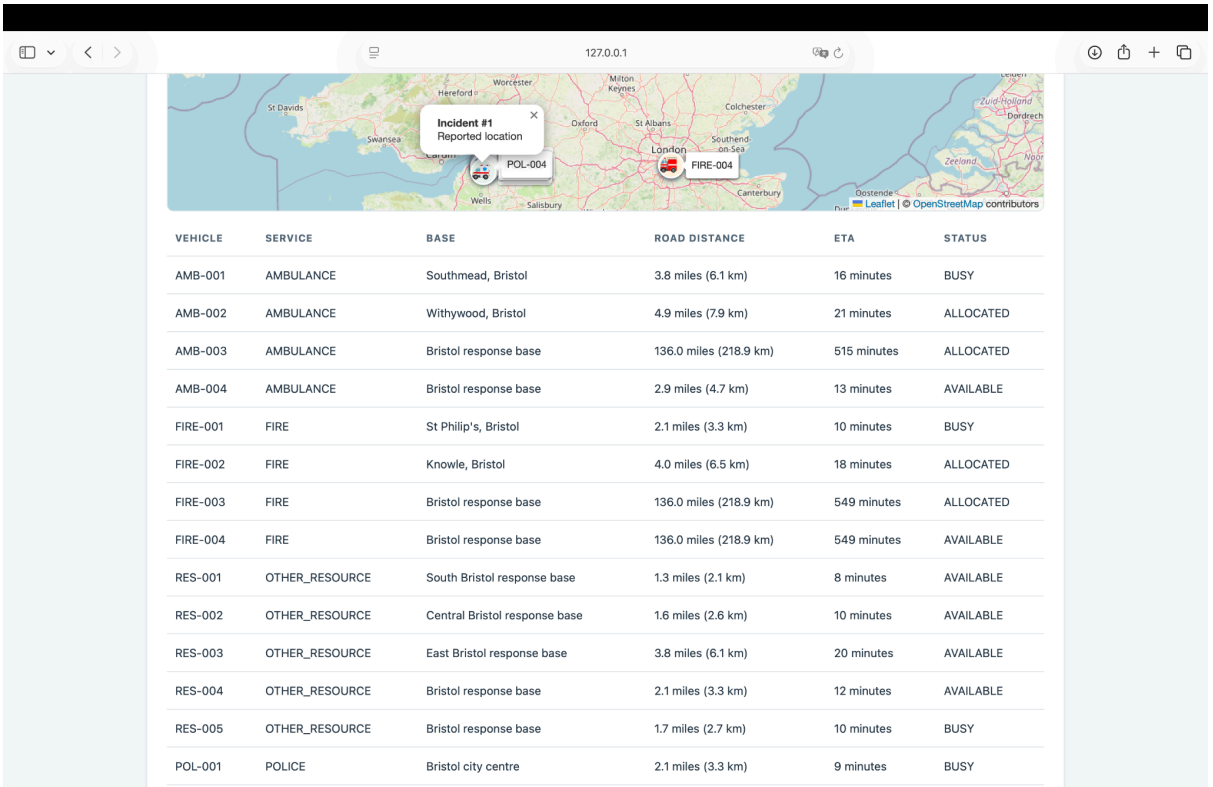
This section contains screenshots of the ERRS application, highlighting its main features, user dashboards and incident management workflow.



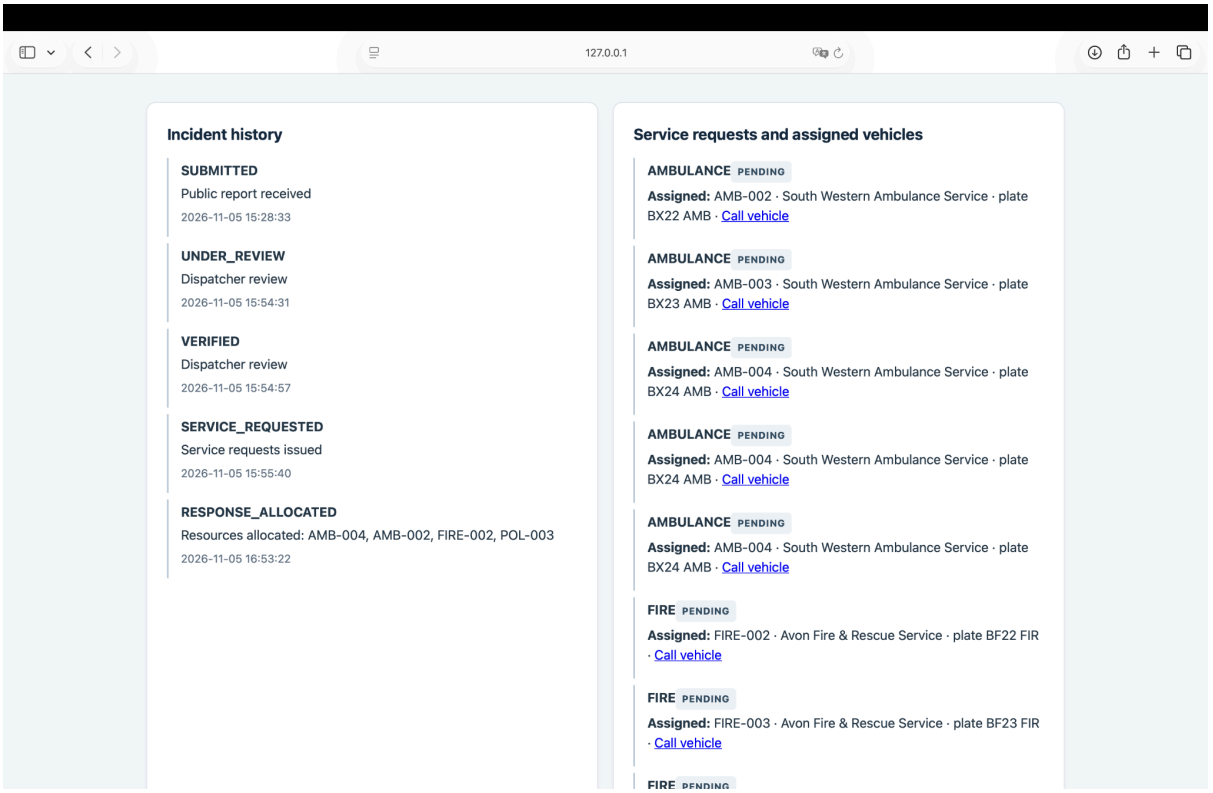

127.0.0.1
E
ERRS
Emergency Rapid Response System
Home
Report
Track report
Staff login
DEMO PROTOTYPE
REPORTER ACCESS
Track your report
Enter either your reference number or your private access token. You do not need both.
Reference number (optional)
Private access token (optional)
Use one of these identifiers to view public-safe updates.
Check status
ERRS · Fictional educational prototype
Simulated regions: North · Central · South


127.0.0.1
E
ERRS
Emergency Rapid Response System
Home
Report
Track report
Staff login
DEMO PROTOTYPE
STAFF PORTAL
Staff sign in
Use an account created by the project administrator.
Username
Password
Sign in
ERRS · Fictional educational prototype
Simulated regions: North · Central · South





VEHICLE	SERVICE	BASE	ROAD DISTANCE	ETA	STATUS
AMB-001	AMBULANCE	Southmead, Bristol	3.8 miles (6.1 km)	16 minutes	BUSY
AMB-002	AMBULANCE	Withywood, Bristol	4.9 miles (7.9 km)	21 minutes	ALLOCATED
AMB-003	AMBULANCE	Bristol response base	136.0 miles (218.9 km)	515 minutes	ALLOCATED
AMB-004	AMBULANCE	Bristol response base	2.9 miles (4.7 km)	13 minutes	AVAILABLE
FIRE-001	FIRE	St Philip's, Bristol	2.1 miles (3.3 km)	10 minutes	BUSY
FIRE-002	FIRE	Knowle, Bristol	4.0 miles (6.5 km)	18 minutes	ALLOCATED
FIRE-003	FIRE	Bristol response base	136.0 miles (218.9 km)	549 minutes	ALLOCATED
FIRE-004	FIRE	Bristol response base	136.0 miles (218.9 km)	549 minutes	AVAILABLE
RES-001	OTHER_RESOURCE	South Bristol response base	1.3 miles (2.1 km)	8 minutes	AVAILABLE
RES-002	OTHER_RESOURCE	Central Bristol response base	1.6 miles (2.6 km)	10 minutes	AVAILABLE
RES-003	OTHER_RESOURCE	East Bristol response base	3.8 miles (6.1 km)	20 minutes	AVAILABLE
RES-004	OTHER_RESOURCE	Bristol response base	2.1 miles (3.3 km)	12 minutes	AVAILABLE
RES-005	OTHER_RESOURCE	Bristol response base	1.7 miles (2.7 km)	10 minutes	BUSY
POL-001	POLICE	Bristol city centre	2.1 miles (3.3 km)	9 minutes	BUSY


127.0.0.1
Incident history
SUBMITTED
Public report received
2026-11-05 15:28:33
UNDER_REVIEW
Dispatcher review
2026-11-05 15:54:31
VERIFIED
Dispatcher review
2026-11-05 15:54:57
SERVICE_REQUESTED
Service requests issued
2026-11-05 15:55:40
RESPONSE_ALLOCATED
Resources allocated: AMB-004, AMB-002, FIRE-002, POL-003
2026-11-05 16:53:22
Service requests and assigned vehicles
AMBULANCE PENDING
Assigned: AMB-002 · South Western Ambulance Service · plate BX22 AMB · Call vehicle
AMBULANCE PENDING
Assigned: AMB-003 · South Western Ambulance Service · plate BX23 AMB · Call vehicle
AMBULANCE PENDING
Assigned: AMB-004 · South Western Ambulance Service · plate BX24 AMB · Call vehicle
AMBULANCE PENDING
Assigned: AMB-004 · South Western Ambulance Service · plate BX24 AMB · Call vehicle
AMBULANCE PENDING
Assigned: AMB-004 · South Western Ambulance Service · plate BX24 AMB · Call vehicle
FIRE PENDING
Assigned: FIRE-002 · Avon Fire & Rescue Service · plate BF22 FIR · Call vehicle
FIRE PENDING
Assigned: FIRE-003 · Avon Fire & Rescue Service · plate BF23 FIR · Call vehicle

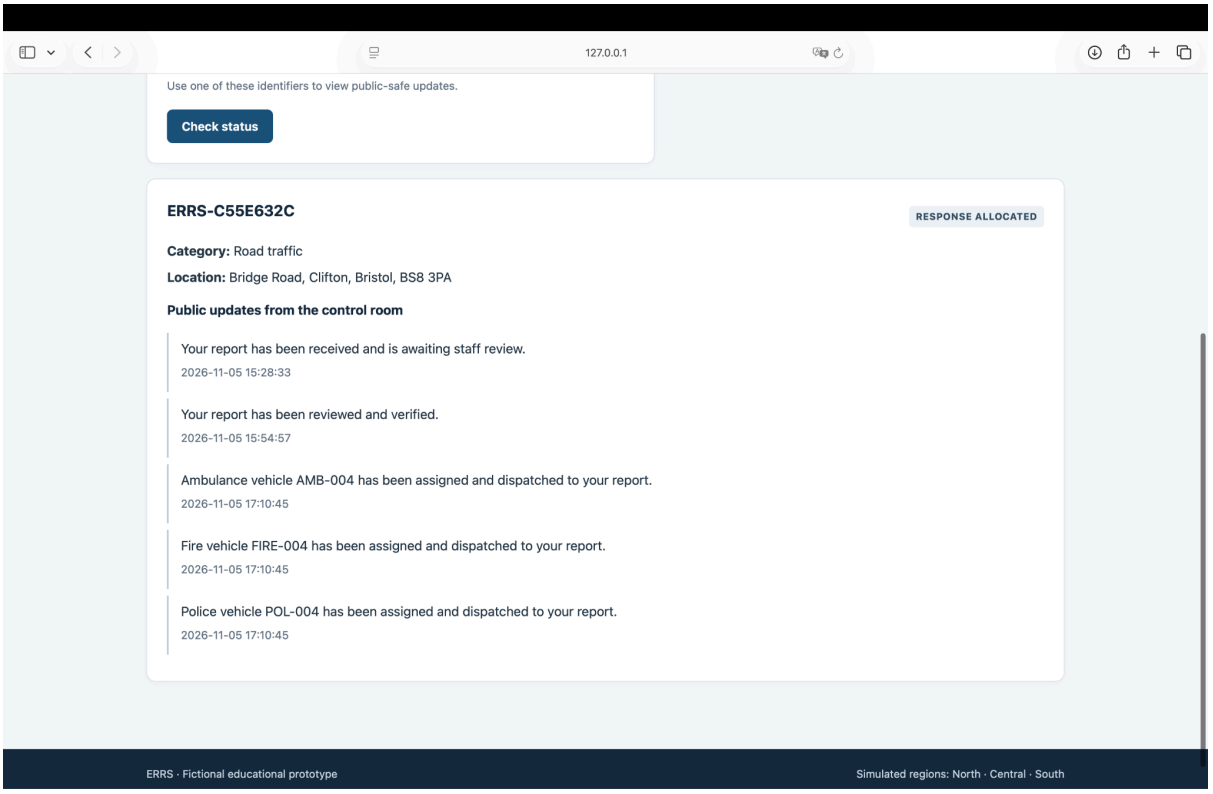

127.0.0.1
Use one of these identifiers to view public-safe updates.
Check status
ERRS-C55E632C
RESPONSE ALLOCATED
Category: Road traffic
Location: Bridge Road, Clifton, Bristol, BS8 3PA
Public updates from the control room
Your report has been received and is awaiting staff review.
2026-11-05 15:28:33
Your report has been reviewed and verified.
2026-11-05 15:54:57
Ambulance vehicle AMB-004 has been assigned and dispatched to your report.
2026-11-05 17:10:45
Fire vehicle FIRE-004 has been assigned and dispatched to your report.
2026-11-05 17:10:45
Police vehicle POL-004 has been assigned and dispatched to your report.
2026-11-05 17:10:45
ERRS · Fictional educational prototype
Simulated regions: North · Central · South

Figure 2 - Security Evidence – Access Control and Policy Tests

```
(.venv) darshan@Darshans-MacBook-Air ERRS_Coursework_Submission % python3 -m pytest tests/test_application.py -v
tests/test_application.py::test_public_report_and_private_tracking PASSED
tests/test_application.py::test_report_valida
tests/test_application.py::test_login_and_ses
tests/test_application.py::test_public_routes
tests/test_application.py::test_authorised_st
tests/test_application.py::test_report_page_signs_out_staff_session PASSED
tests/test_application.py::test_home_and_tracking_sign_out_staff_session PASSED
tests/test_application.py::test_tracking_shows_public_updates_only PASSED
tests/test_application.py::test_public_updates_explain_arrival_and_authority_contact PASSED
tests/test_application.py::test_password_hashing PASSED
tests/test_application.py::test_assessment_and_classification PASSED
tests/test_application.py::test_views_do_not_mutate_source PASSED
tests/test_application.py::test_policy_scenarios PASSED
tests/test_application.py::test_nine_consensus_experiments PASSED
tests/test_application.py::test_policy_read_and_write_directions PASSED
tests/test_application.py::test_dataset_classification_and_research_view PASSED
tests/test_application.py::test_retention_blocks_destruction_on_hold PASSED
tests/test_application.py::test_invalid_transition_and_checkpoint PASSED
tests/test_application.py::test_retention_legal_hold PASSED
tests/test_application.py::test_audit_chain_after_login PASSED
tests/test_application.py::test_governance_scenarios_do_not_relax_policy PASSED
tests/test_application.py::test_offline_region_does_not_invent_allocation PASSED
tests/test_application.py::test_validator_admission_revocation_and_rotation PASSED
```

Navigate to Command
Scroll to Previous Command (⌘↑)
Scroll to Next Command (⌘↓)

rtal PASSED

dispatcher_only_access PASSED

DATA GOVERNANCE

Source datasets

Imported coursework records. Dataset availability depends on your staff role.

DATASET	RECORDS IMPORTED	LAST IMPORT	
incidents.json	0	Not imported	View records
dispatch.json	0	Not imported	View records
responders.json	0	Not imported	View records

Figure 3 - Consensus Protocol Testing Evidence

The system completed nine consensus experiments covering BFT, Paxos and Raft. All nineexperimentspassed, demonstrating the handling of different faults and security threats.

```
(.venv) darshan@Darshans-MacBook-Air ERRS_Coursework_Submission % cat evidence/test-results/experiment-results.txt
BFT-01: DETECTED (PASS)
BFT-02: REJECTED (PASS)
BFT-03: MITIGATED (PASS)
PAXOS-04: SAFE_STOP (PASS)
PAXOS-05: NO_QUORUM (PASS)
PAXOS-06: REJECTED (PASS)
RAFT-07: NO_QUORUM (PASS)
RAFT-08: DEGRADED (PASS)
RAFT-09: DEGRADED (PASS)
E1: DENY
E2: DENY
E3: NO_NEW_COMMIT
E4: MITIGATED
E5: Regulations require emergency-response records to be retained for twenty years
Evidence written under /Users/darshan/Downloads/ProjectDemo/ERRS_Coursework_Submission/evidence
(.venv) darshan@Darshans-MacBook-Air ERRS_Coursework_Submission %
```

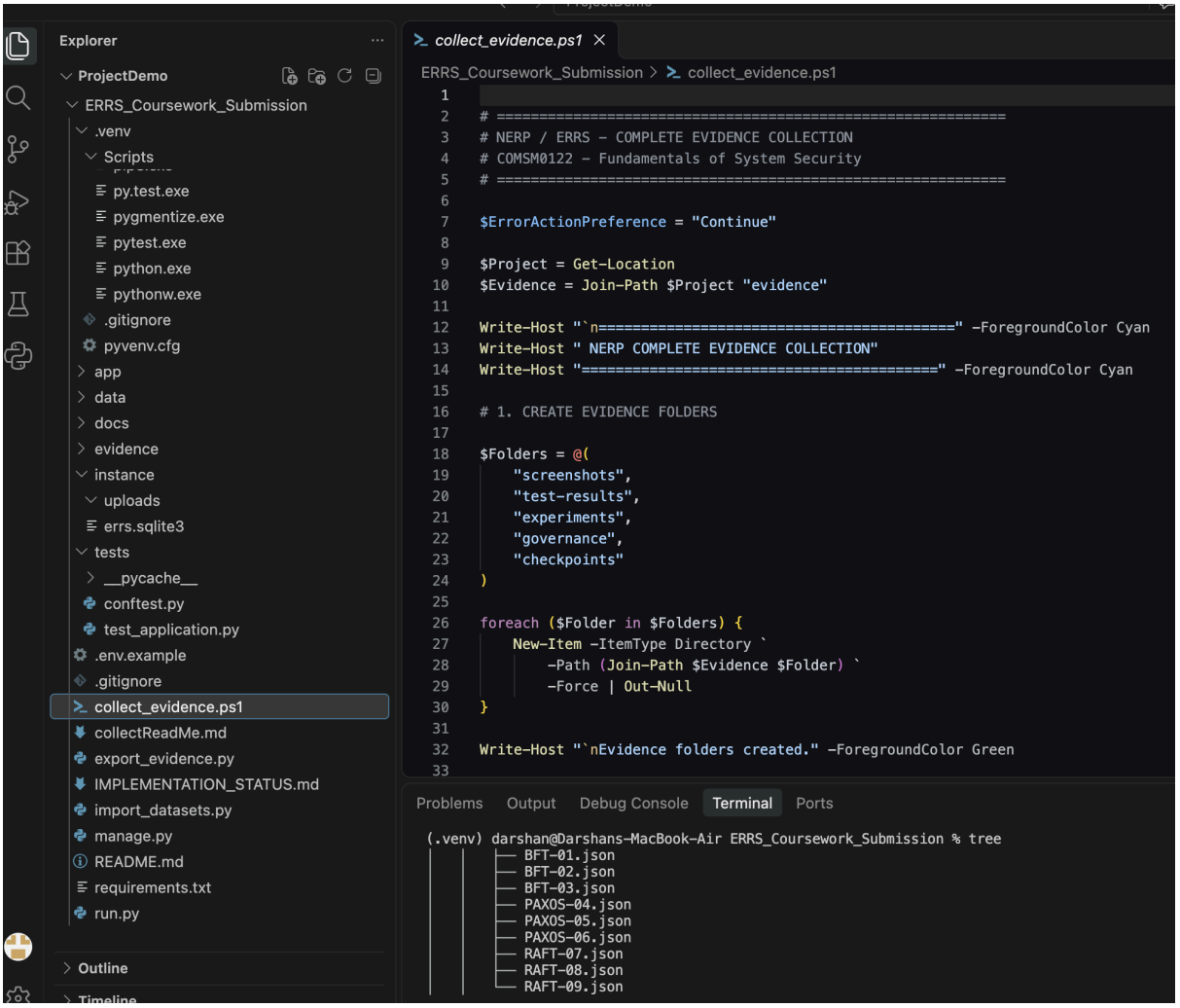


Figure 4 - Classification Dashboard

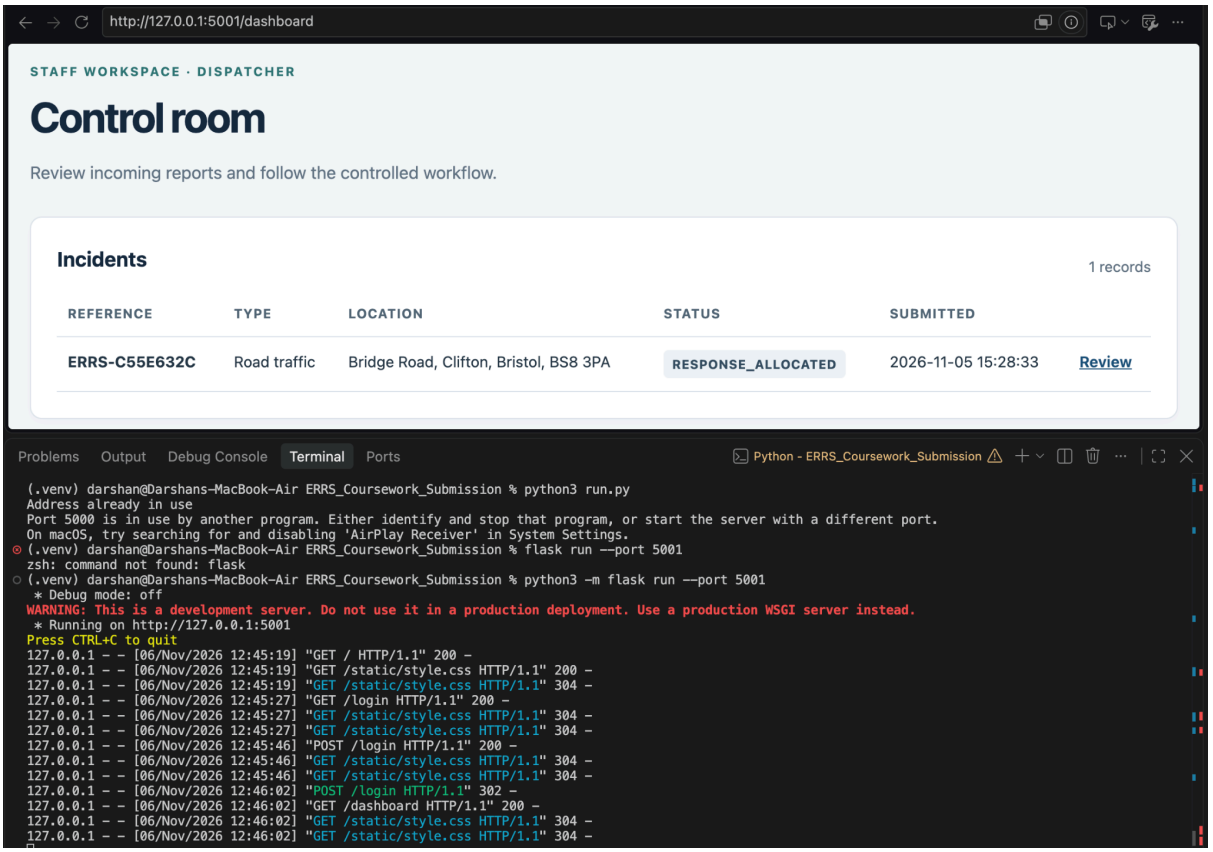


Figure 5 - Dataset Views

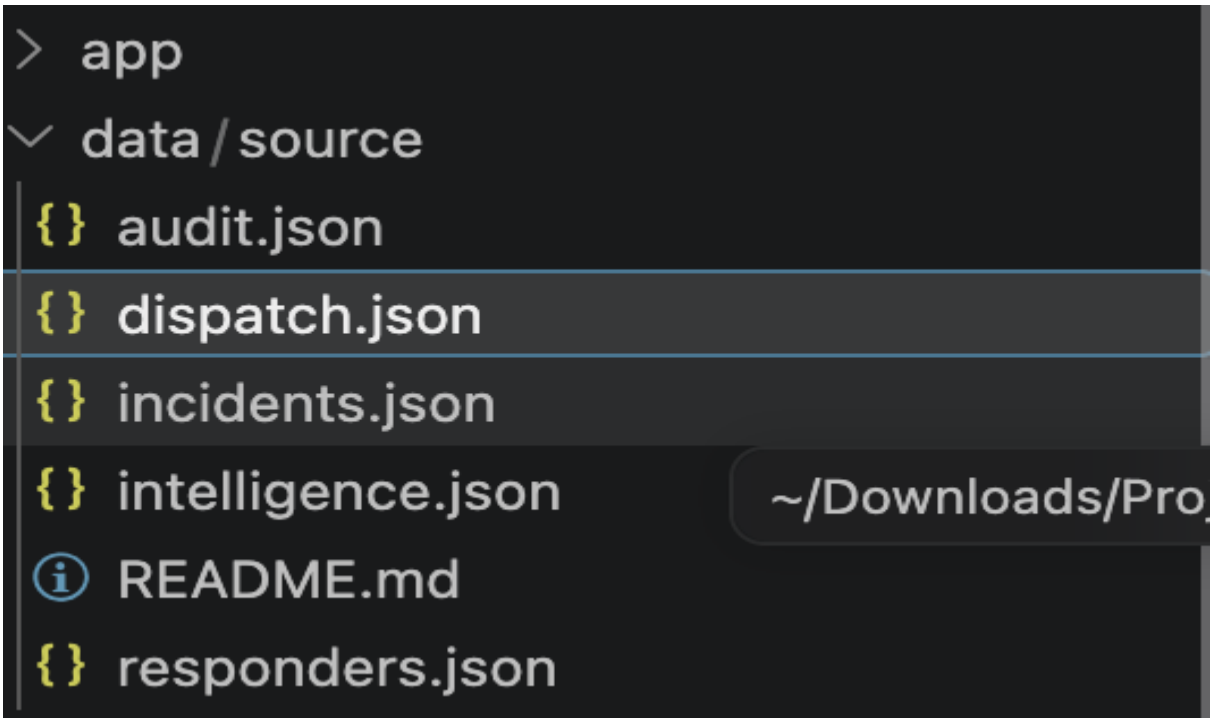


Figure 6 - Governance, Security, Classification and Retention, Evidence.

```
(.venv) darshan@Darshans-MacBook-Air ERRS_Coursework_Submission % cat run.py
cat app/__init__.py
cat app/governance.py
from app import create_app
app = create_app()
if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=True)
import os
from pathlib import Path
from flask import Flask
from .database import init_db
from .routes import register_routes

def create_app(test_config=None):
    app = Flask(__name__, instance_relative_config=True)
    app.config.from_mapping(
        SECRET_KEY=os.environ.get("SECRET_KEY", "development-only-change-this-key"),
        DATABASE=os.environ.get("DATABASE_PATH", str(Path(app.instance_path) / "errs.sqlite3")),
        MAX_CONTENT_LENGTH=25 * 1024 * 1024,
        MEDIA_UPLOAD_FOLDER=str(Path(app.instance_path) / "uploads"),
        SESSION_COOKIE_HTTPONLY=True,
        SESSION_COOKIE_SAMESITE="Lax",
        SESSION_COOKIE_SECURE=False,
```

```
Problems  Output  Debug Console  Terminal  Ports                                   zsh - ERRS_Coursework_Submission

(.venv) darshan@Darshans-MacBook-Air ERRS_Coursework_Submission % cat run.py
cat app/__init__.py
cat app/governance.py
    held = retention_decision("RESTRICTED", legal_hold=True, age_years=25)
    return {
        "id": "E5",
        "title": "Regulations require emergency-response records to be retained for twenty years",
        "datasets": ["incidents.json", "dispatch.json", "responders.json", "intelligence.json", "audit.json"],
        "properties": ["confidentiality", "storage", "integrity"],
        "within_period": within,
        "at_expiry": expired,
        "legal_hold": held,
        "explanation": "RESTRICTED uses 20 years. PUBLIC, OPERATIONAL and SENSITIVE keep shorter demonstration periods. A legal hold blocks destruction. O
riginal files in data/source are not deleted.",
    }


def all_scenarios():
    return [
        research_organisation_request(),
        responder_classified_request(),
        regional_datacentre_loss(),
        audit_validator_compromise(),
        twenty_year_retention(),
    ]
(.venv) darshan@Darshans-MacBook-Air ERRS_Coursework_Submission %
```

```
    def all_scenarios():
        return [
            research_organisation_request(),
            responder_classified_request(),
            regional_datacentre_loss(),
            audit_validator_compromise(),
            twenty_year_retention(),
        ]
(.venv) darshan@Darshans-MacBook-Air ERRS_Coursework_Submission % cat app/governance.py
cat app/security.py
cat app/engines.py
cat app/templates/governance.html
cat app/templates/security.html
```

Figure 7 - Dataset and retention implementation.

```
 dispatch.json and audit.json. Original files should remain unchanged. The classification API creates derived
section>
{% endblock %}
{% extends "base.html" %}{% block title %}Security demonstrations{% endblock %}{% block content %}
<div class="page-heading"><div class="eyebrow">SECURITY POLICY & CONSENSUS</div><h1>Security demonstrations</
t-injection teaching simulations.</p></div>
<section class="panel"><h2>Combined policy scenarios</h2>{% for r in results %}<div class="scenario"><div><st
ation'] }}</p><small>Failed checks: {{ r['failed_rules'] }}</small></div><span class="status {{ 'good' if r['
ision'] }}</span></div>{% endfor %}</section>
<section class="panel"><h2>Nine consensus experiments</h2><p>Each action runs a deterministic simulation and
hing models, not production consensus implementations.</p>
{% for group, ids in [('BFT',['BFT-01','BFT-02','BFT-03']),('Paxos',['PAXOS-04','PAXOS-05','PAXOS-06']),('Raf
 group }}</h3><div class="experiment-grid">{% for id in ids %}<form method="post" action="{{ url_for('experime
an><button class="button secondary">Run experiment</button></form>{% endfor %}</div>{% endfor %}</section>
{% endblock %}
(.venv) darshan@Darshans-MacBook-Air ERRS_Coursework_Submission % cat app/engines.py
cat app/templates/datasets.html
cat app/templates/dataset_records.html
cat app/templates/governance.html
```

Classification and Disclosure Table

Level	Prototype handling	Disclosure consideration
PUBLIC	**Low-sensitivity information.**	**Check field content before external release.**
OPERATIONAL	**Internal coordination and dispatch fields.**	**Locations and assignments can expose operational activity.**
SENSITIVE	**Caller names, phone numbers and personal or medical information.**	**Disclosure can identify callers or reveal private information.**
RESTRICTED	**Intelligence and security-sensitive information.**	**Disclosure may expose intelligence sources or operational risks.**

A record takes the highest applicable field or hint classification. Figure C1 shows the observed distribution across all five datasets. All 20 incident records were SENSITIVE. Dispatch and audit records were OPERATIONAL, while responder and intelligence records spanned more than one level.

Figure C1. Classification counts from authenticated requests for all five datasets.

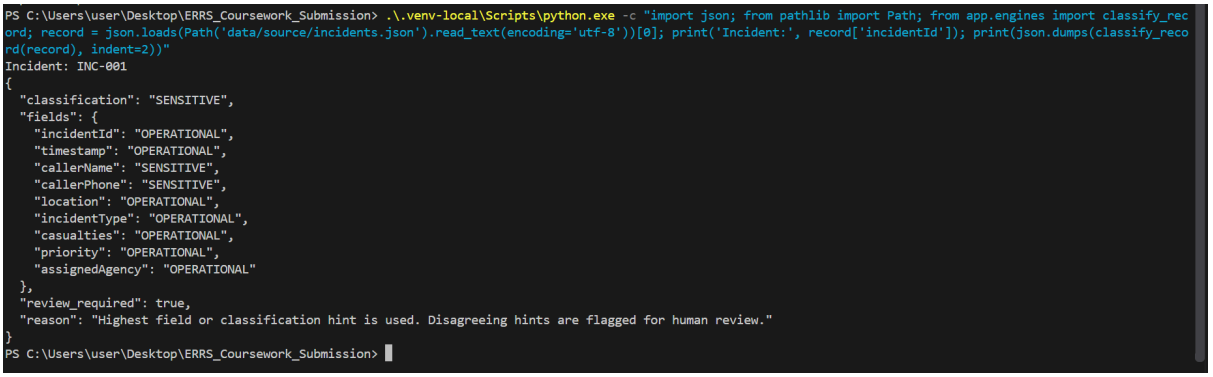
```
PS C:\Users\user\Desktop\ERRS_Coursework_Submission> .\.venv-local\Scripts\python.exe import_datasets.py
incidents.json: imported 20 records
dispatch.json: imported 20 records
responders.json: imported 20 records
intelligence.json: imported 20 records
audit.json: imported 20 records
Import complete: 100 records across five datasets.
PS C:\Users\user\Desktop\ERRS_Coursework_Submission>
```

An unauthenticated request was redirected to sign-in. The authorised auditor received HTTP 200 for each dataset. These checks cover the tested access path and role.

C2 Classification engine

The application classifies each record in app/engines.py. The engine normalises field names, checks known fields and keywords, and maps metadata such as classificationHint and clearanceLevel to its four levels.

Figure C2. Field labels and review flag returned for incident INC-001



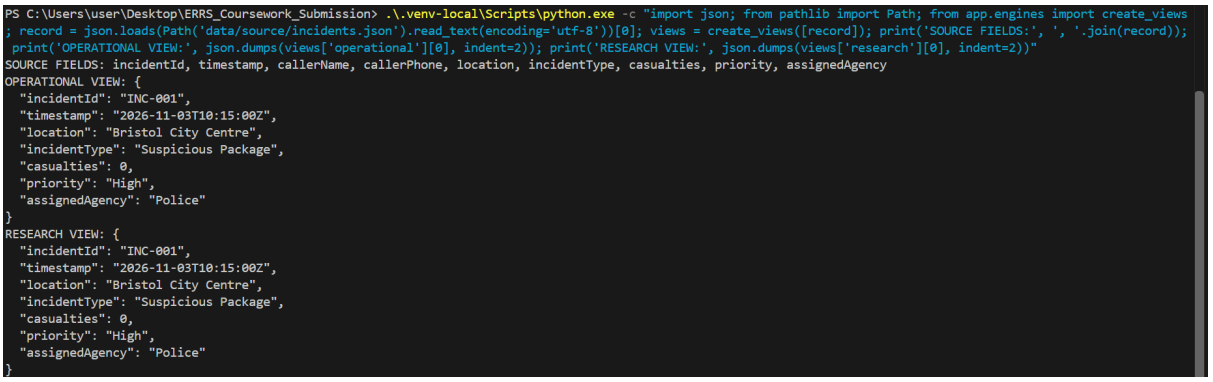
Caller name and phone fields received SENSITIVE labels. The incident type and location remained OPERATIONAL. The record received SENSITIVE overall, and review_required was true. The API returned the field map and its standard explanation alongside the record label.

The review flag helps identify records for human assessment. Keyword matching and broad metadata mappings still require judgement. Unrecognised fields can fall back to PUBLIC, and the importer's stored metadata and the runtime field labels can differ.

C3 Information flow

The operational view selects PUBLIC and OPERATIONAL fields. The research view applies additional key filtering to this selection and masks locations containing digits. The field comparison below uses the original incident and the corresponding API outputs.

Figure C3a. Field presence in the source, operational view and research view.



INC-001 retains its ID, timestamp and Bristol City Centre location in the research view. The direct caller identifiers are absent, but the remaining fields create a linkage risk. The prototype provides partial sanitisation.

Figure C3b. Analytics counts, one category entry and the source integrity comparison.

```
PS C:\Users\user\Desktop\ERRS_Coursework_Submission> .\.venv-local\Scripts\python.exe -c "import json; from pathlib import Path; from app.engines import create_views
; records = json.loads(Path('data/source/incidents.json').read_text(encoding='utf-8')); analytics = create_views(records)['analytics']; print('ANALYTICS VIEW - incid
ents.json'); print('Records:', len(records)); print('Counts by classification:', analytics['counts_by_classification']); print('First category entry:', json.dumps(an
alytics['categories'][0], indent=2))"
ANALYTICS VIEW - incidents.json
Records: 20
Counts by classification: {'PUBLIC': 0, 'OPERATIONAL': 0, 'SENSITIVE': 20, 'RESTRICTED': 0}
First category entry: {
  "category": "Suspicious Package",
  "classification": "SENSITIVE",
  "area": "Bristol City Centre"
}
PS C:\Users\user\Desktop\ERRS_Coursework_Submission>
```

Analytics keeps classification counts and a category and area entry for each incident. Direct caller details are omitted. Area and category can still disclose context. All five source-file SHA-256 values matched before and after the import and API checks, which demonstrates source preservation for this run.

C4 Retention and governance

The prototype uses retention periods of two, seven, ten and twenty years for PUBLIC, OPERATIONAL, SENSITIVE and RESTRICTED data respectively. The periods are demonstration policy settings. The comparison below holds the classification constant for the expiry and legal-hold examples.

Figure C4a. Return values for four classification levels and an expiry case with and without a hold.

```
RETENTION YEARS: {'PUBLIC': 2, 'OPERATIONAL': 7, 'SENSITIVE': 10, 'RESTRICTED': 20}
Age 1 year: {'retention_years': 2, 'action': 'RETAIN', 'archive_status': 'ACTIVE', 'deletion_status': 'NOT_DUE', 'reason': 'Record remains within its classification
retention period.'}
Age 3 years: {'retention_years': 2, 'action': 'DESTROY', 'archive_status': 'ARCHIVED', 'deletion_status': 'DESTROYED', 'reason': 'Retention period has elapsed and no
 legal hold is set. Derived copies may be destroyed; source coursework files are left unchanged.'}
Age 3 years with legal hold: {'retention_years': 2, 'action': 'HOLD', 'archive_status': 'ACTIVE', 'deletion_status': 'BLOCKED', 'reason': 'Legal hold prevents deleti
on regardless of expiry.'}
PS C:\Users\user\Desktop\ERRS_Coursework_Submission>
```

An expired PUBLIC record returned DESTROY. The same input with a hold returned HOLD and BLOCKED. ARCHIVED and DESTROYED are decision fields returned by the helper. The prototype leaves the source files in place. Actual archive migration, secure erasure and backup expiry are outside the demonstrated behaviour.

The existing classification, view and retention tests were run against the ProjectDemo source. Five selected tests passed, and the remaining eighteen tests were outside this selection.

Figure C4b. Actual pytest output for the selected classification, view and retention tests.

```
PS C:\Users\user\Desktop\ERRS_Coursework_Submission> .\.venv-local\Scripts\python.exe -m pytest -v tests/test_application.py -k "classification or views or retention
"
============================================================ test session starts ============================================================
platform win32 -- Python 3.12.10, pytest-8.4.2, pluggy-1.7.0 -- C:\Users\user\Desktop\ERRS_Coursework_Submission\.venv-local\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\Desktop\ERRS_Coursework_Submission
collected 17 items / 12 deselected / 5 selected

tests/test_application.py::test_assessment_and_classification PASSED                                                                  [ 20%]
tests/test_application.py::test_views_do_not_mutate_source PASSED                                                                     [ 40%]
tests/test_application.py::test_dataset_classification_and_research_view PASSED                                                       [ 60%]
tests/test_application.py::test_retention_blocks_destruction_on_hold PASSED                                                           [ 80%]
tests/test_application.py::test_retention_legal_hold PASSED                                                                           [100%]

========================================================== 5 passed, 12 deselected in 0.04s ==========================================================
PS C:\Users\user\Desktop\ERRS_Coursework_Submission>
```

Reproduction command

python -m pytest -v tests/test_application.py -k "classification or views or retention"

Part D : Multi-Layer Security Policy Engine

1. **Scope And Evidence Approach**

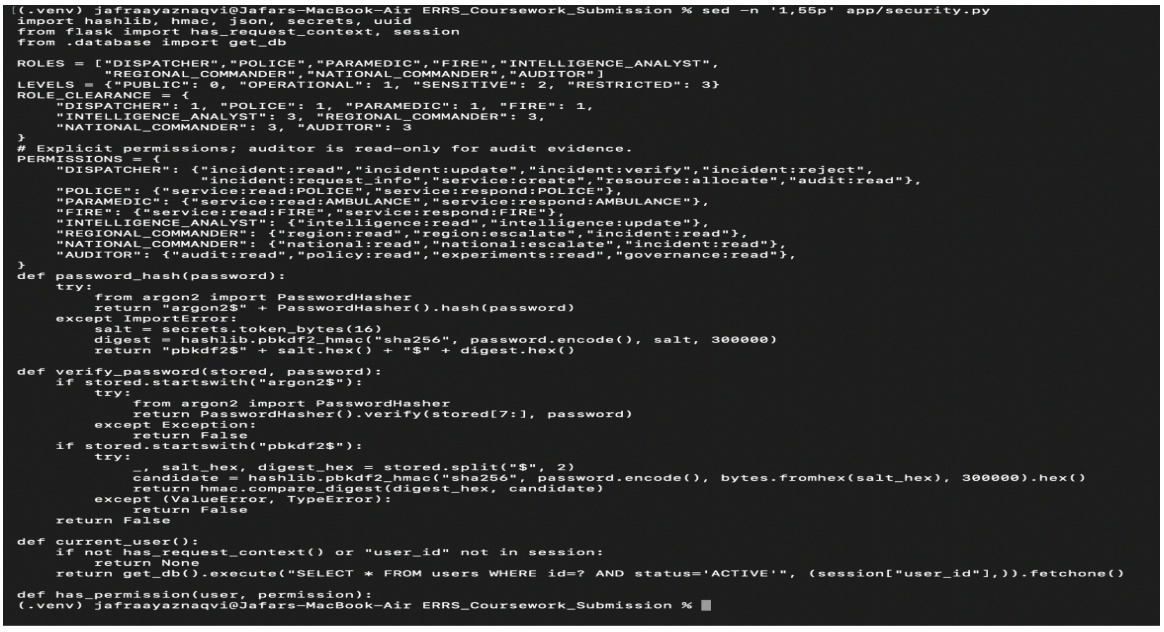
   Part D evaluates a multilayer policy in which role permissions, confidentiality, classification, integrity classification and contextual conditions are checked before access permitted. The implementation is in app/security.py and policy assertions are in tests/test_application.py .

   The prototype defines four ordered levels :- PUBLIC, OPERATIONAL, SENSITIVE AND RESTRICTED.

   The combined evaluator checks ACM, Bell-LaPadula, Bibla and CONTEXT. A request is permitted only when no check fails.

2. **Access Control Matrix (ACM)**

   The system has eight different user roles: DISPATCHER, POLICE, PARAMEDIC, FIRE, INTELLIGENCE_ANALYST, REGIONAL_COMMANDER, NATIONAL_COMMANDER and AUDITOR. Each role has its own set of permissions, which determines what actions a user can perform. The has_permission() function checks whether a user is allowed to carry out a particular action based on their assigned role. The system also uses four security levels: PUBLIC, OPERATIONAL, SENSITIVE and RESTRICTED. Each role is assigned a clearance level through ROLE_CLEARANCE, which helps control access to information. The AUDITOR role is limited to read-only access for audit records, security policies, experiments and governance evidence.



```
(.venv) jafraayaznaqvi@Jafars-MacBook-Air ERRS_Coursework_Submission % sed -n '1,55p' app/security.py
import hashlib, hmac, json, secrets, uuid
from flask import has_request_context, session
from .database import get_db

ROLES = ["DISPATCHER","POLICE","PARAMEDIC","FIRE","INTELLIGENCE_ANALYST",
         "REGIONAL_COMMANDER","NATIONAL_COMMANDER","AUDITOR"]
LEVELS = {"PUBLIC": 0, "OPERATIONAL": 1, "SENSITIVE": 2, "RESTRICTED": 3}
ROLE_CLEARANCE = {
    "DISPATCHER": 1, "POLICE": 1, "PARAMEDIC": 1, "FIRE": 1,
    "INTELLIGENCE_ANALYST": 3, "REGIONAL_COMMANDER": 3,
    "NATIONAL_COMMANDER": 3, "AUDITOR": 3
}
# Explicit permissions; auditor is read-only for audit evidence.
PERMISSIONS = {
    "DISPATCHER": {"incident:read","incident:update","incident:verify","incident:reject",
                   "incident:request_info","service:create","resource:allocate","audit:read"},
    "POLICE": {"service:read:POLICE","service:respond:POLICE"},
    "PARAMEDIC": {"service:read:AMBULANCE","service:respond:AMBULANCE"},
    "FIRE": {"service:readsFIRE","service:respond:FIRE"},
    "INTELLIGENCE_ANALYST": {"intelligence:read","intelligence:update"},
    "REGIONAL_COMMANDER": {"region:read","region:escalate","incident:read"},
    "NATIONAL_COMMANDER": {"national:read","national:escalate","incident:read"},
    "AUDITOR": {"audit:read","policy:read","experiments:read","governance:read"},
}
def password_hash(password):
    try:
        from argon2 import PasswordHasher
        return "argon2$" + PasswordHasher().hash(password)
    except ImportError:
        salt = secrets.token_bytes(16)
        digest = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 300000)
        return "pbkdf2$" + salt.hex() + "$" + digest.hex()

def verify_password(stored, password):
    if stored.startswith("argon2$"):
        try:
            from argon2 import PasswordHasher
            return PasswordHasher().verify(stored[7:], password)
        except Exception:
            return False
    if stored.startswith("pbkdf2$"):
        try:
            _, salt_hex, digest_hex = stored.split("$", 2)
            candidate = hashlib.pbkdf2_hmac("sha256", password.encode(), bytes.fromhex(salt_hex), 300000).hex()
            return hmac.compare_digest(digest_hex, candidate)
        except (ValueError, TypeError):
            return False
    return False

def current_user():
    if not has_request_context() or "user_id" not in session:
        return None
    return get_db().execute("SELECT * FROM users WHERE id=? AND status='ACTIVE'", (session["user_id"],)).fetchone()

def has_permission(user, permission):
(.venv) jafraayaznaqvi@Jafars-MacBook-Air ERRS_Coursework_Submission %
```

3. **Bell-LaPadula Confidentiality**

   The code applies the Bell-La Padula read rule as subject level => resource level (No Read Up) and the write rule as subject <= resource level (No

write Down). The test checks that a dispatcher cannot read as RESTRICTED object and that a dispatcher write to PUBLIC object is denied by BLP.

```
(.venv) jafraayaznaqvi@Jafars-MacBook-Air ERRS_Coursework_Submission % sed -n '120,155p' tests/test_application.py
    views=create_views(source)
    assert "operational" in views and "analytics" in views and "research" in views
    assert json.dumps(source,sort_keys=True)==before

def test_policy_scenarios():
    dispatcher={"role":"DISPATCHER","status":"ACTIVE"}
    denied=evaluate_policy(dispatcher,"incident:read","RESTRICTED","RESTRICTED",2,2)
    assert denied["decision"]=="DENY"
    permitted=evaluate_policy(dispatcher,"incident:read","OPERATIONAL","OPERATIONAL",2,2)
    assert permitted["decision"]=="PERMIT"
    biba_denied=evaluate_policy({"role":"INTELLIGENCE_ANALYST","status":"ACTIVE"},
                                "intelligence:read","SENSITIVE","PUBLIC",2,2)
    assert "BIBA" in biba_denied["failed_rules"]

def test_nine_consensus_experiments():
    ids=[f"BFT-0{i}" for i in range(1,4)]+[f"PAXOS-0{i}" for i in range(4,7)]+[f"RAFT-0{i}" for i in range(7,10)]
    results=[run_experiment(i) for i in ids]
    assert len(results)==9
    assert all(result["test_result"]=="PASS" and result["committed_under_fault"] is False for result in results)
    assert all(len(result["execution_log"])>=3 for result in results)
    by_id={result["id"]: result["observed_result"] for result in results}
    assert by_id["BFT-01"]=="DETECTED"
    assert by_id["PAXOS-05"]=="NO_QUORUM"
    assert by_id["RAFT-07"]=="NO_QUORUM"

def test_policy_read_and_write_directions():
    dispatcher={"role":"DISPATCHER","status":"ACTIVE"}
    write_down=evaluate_policy(dispatcher,"incident:update","PUBLIC","OPERATIONAL",2,2)
    assert write_down["decision"]=="DENY" and "BELL_LAPADULA" in write_down["failed_rules"]
    write_up=evaluate_policy({"role":"DISPATCHER","status":"ACTIVE"},"incident:update","OPERATIONAL","RESTRICTED",2,2)
    assert "BIBA" in write_up["failed_rules"]

def test_dataset_classification_and_research_view():
    incident={"callerName":"Alice Turner","callerPhone":"07700110001","incidentType":"Fire","location":"Taunton","casualties":0}
    assert classify_record(incident)["classification"]=="SENSITIVE"
    intel={"classificationHint":"TOP SECRET","summary":"Credible attack planning","suspectIdentifier":"SUBJ-501","threatLevel":"CRITICAL"}
(.venv) jafraayaznaqvi@Jafars-MacBook-Air ERRS_Coursework_Submission %
```

Figure :- Test source showing policy scenarios and read/write direction assertions.

4. **Bibla Integrity**

The code applies strict Bibla rules. For reads, subject level =< integrity level (No Read Down), for writes, subject >= integrity level (No Write Up). The tests include an intelligence analyst reading a SENSITIVE object with PUBLIC integrity, which is denied by Bibla and a dispatcher write up case that fails Bibla.

```
(.venv) jafraayaznaqvi@Jafars-MacBook-Air ERRS_Coursework_Submission % python3 -m pytest -v tests/test_application.py -k "policy"
============================= test session starts ==============================
platform darwin -- Python 3.13.5, pytest-8.4.2, pluggy-1.7.0 -- /Library/Frameworks/Python.framework/Versions/3.13/bin/python3
cachedir: .pytest_cache
rootdir: /Users/jafraayaznaqvi/Downloads/ProjectDemo/ERRS_Coursework_Submission
plugins: anyio-4.9.0
collected 23 items / 20 deselected / 3 selected

tests/test_application.py::test_policy_scenarios PASSED                  [ 33%]
tests/test_application.py::test_policy_read_and_write_directions PASSED  [ 66%]
tests/test_application.py::test_governance_scenarios_do_not_relax_policy PASSED [100%]

======================= 3 passed, 20 deselected in 0.01s =======================
(.venv) jafraayaznaqvi@Jafars-MacBook-Air ERRS_Coursework_Submission %
```

Figure :- Focused policy execution :3 passed, 20 deselected.

5. **Combined Policy Scenarios**

The evaluator combines four checks : ACM, BELL_LAPADULLA, BIBLA and CONTEXT. It returns PERMIT only if none of these checks fail. The dashboard displays three deterministic examples :

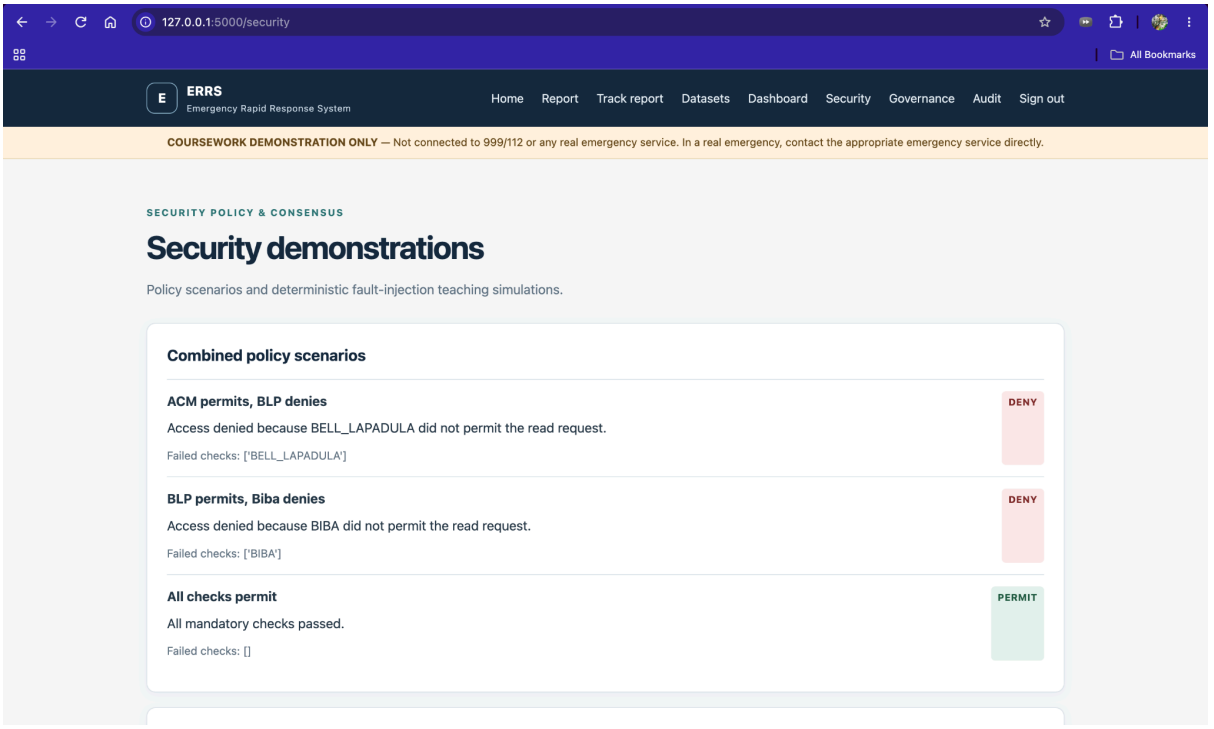


Figure :- Security dashboard showing the three combined policy outcomes.

6. Automated Testing

A fresh test run was executed from the project directory using Python 3.13.5 and pytest 8.4.2. The captured terminal output reports 23 passed in 0.76 seconds.

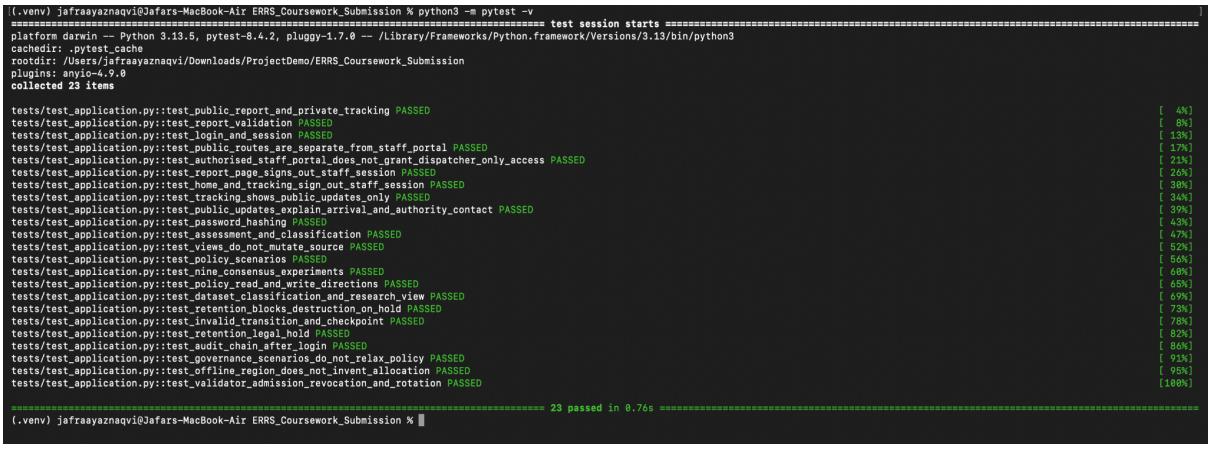


Figure :- Full local pytest run.

7. Audit Integrity And Checkpoint Evidence

The audit implementation links events using a previous-event hash and a SHA-256 hash of the event payload. The verification function recomputes each event and checks the previous-hash linkage. A separate checkpoint uses an HMAC signature derived from the application SECRET_KEY.

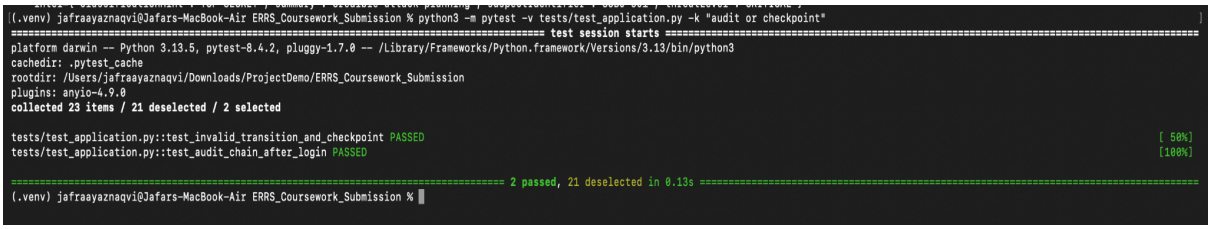


Figure :- Focused audit/checkpoint test

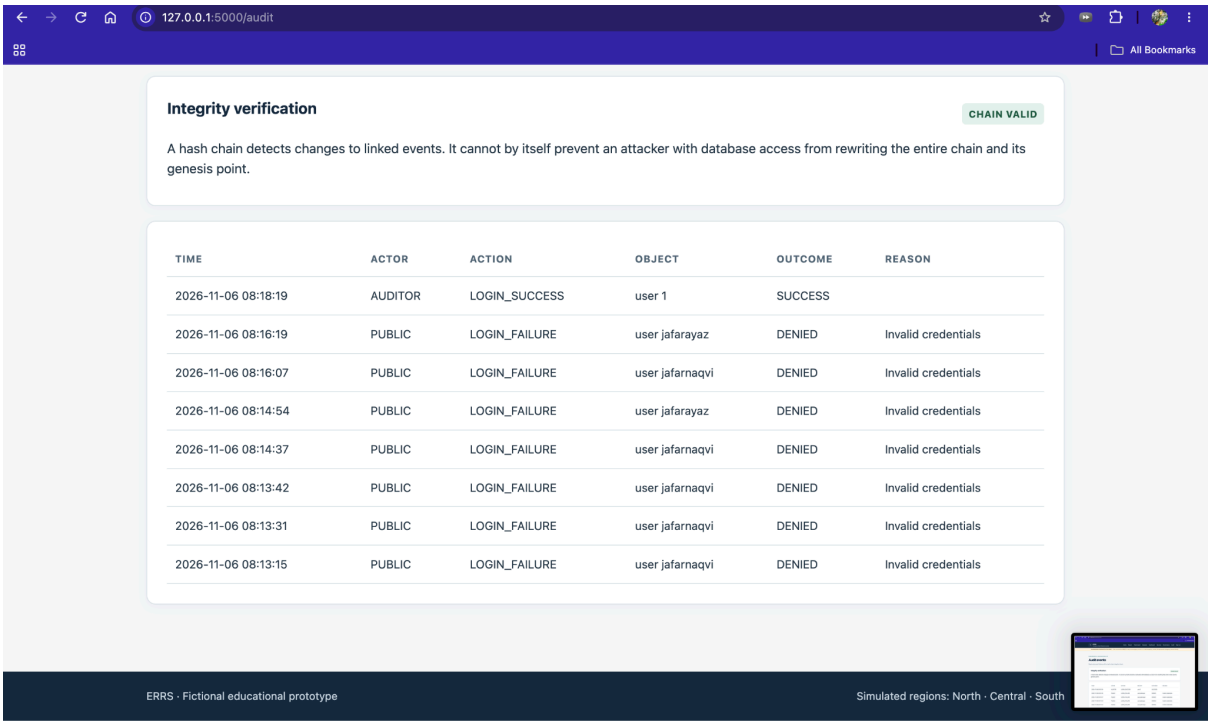


Figure :- Audit events page showing CHAIN VALID and recorded login outcomes.

8. Limitations and Residual Risks

The prototype uses the same role-derived numeric level as both the subject's confidentiality clearance and integrity level.These properties are conceptually distinct; a production design should maintain separate subject clearance and integrity attributes.

The audit page states that the hash chain detects changes to linked events but cannot by itself prevent an attacker with database access from rewriting the entire chain and its genesis point. The checkpoint adds a keyed signature, but its protection depends on SECRET_KEY protection and sufficient separation of the checkpoint from the database threat. The

screenshots and tests are local coursework evidence. They do not establish production readiness, resistance to all attacks, independent penetration-test coverage, or deployment security. The Flask development server is suitable for this local demonstration only.

9. **Assurance Statement**

   The work carried out shows that the prototype brings together different security measures, including role-based access control, Bell–LaPadula, Biba and contextual checks. The three policy scenarios demonstrate how the system handles both permitted and denied access requests. All 23 tests passed during the local test run, and the audit and checkpoint tests also completed successfully. The audit page further confirmed that the hash chain was valid at the time of testing. However, these results only demonstrate that the implemented features worked under the conditions tested. Some limitations still remain, and the prototype should not be considered fully secure or production-ready.

SECTION E: SECURITY GOVERNANCE AND ASSURANCE

E1. Strategic Security Impact Assessment

Once ERRS is deployed, it may face situations that were not part of its original design. Some of these situations could affect the security of information, while others could interrupt emergency response activities. The following five scenarios explain the possible impact on the system.

E1.1 Research Organisation Requests Unrestricted Access to Intelligence Data

Giving a research organisation unrestricted access to intelligence data would create a risk to the confidentiality of stored information. Some records may contain personal details, information about suspects and details of ongoing investigations. If researchers could access all these records, information might be exposed to people who have no operational reason to view it. This would also weaken the purpose of the existing information classification and access control policies. Although access to intelligence data could be useful for research, unrestricted access could affect individual privacy, ongoing investigations and public trust in the system.

E1.2 Emergency Responders Request Access to RESTRICTED Intelligence

There may be situations where emergency responders need information classified as RESTRICTED to deal with a serious incident. Under the current access rules, users with OPERATIONAL clearance cannot access these records. This restriction protects sensitive information, but it could also create difficulties when important information is needed quickly. Allowing access without proper authorisation would increase the risk of information being misused or disclosed. On the other hand, refusing access in every emergency could delay decisions and affect the response. This creates a difficult balance between protecting confidential information and allowing emergency teams to do their work.

E1.3 A Regional Datacentre Becomes Permanently Unavailable

The permanent loss of a regional datacentre could affect the availability of ERRS and make it harder for different regions to coordinate their activities. Incident updates, resource allocation and communication between regions could be disrupted. During our Raft simulation, we observed that the system stops accepting new commitments when a majority of nodes are unavailable. This helps prevent inconsistent data, but it also means that some operations cannot continue. The current project demonstrates this behaviour through simulation. It does not use physically separate datacentres, so the results do not fully represent the problems that could occur during a real infrastructure failure.

E1.4 Validator Credentials Are Compromised

If an attacker obtains a validator's credentials, they could attempt to impersonate that validator and carry out unauthorised operations. This could affect the reliability of consensus decisions and reduce trust in the audit records. During our BFT simulation, we tested credential revocation and replacement using key version checks. However, the prototype still depends on a shared HMAC secret for certain authentication and verification operations. If this secret were exposed, an attacker could potentially misuse it. This is a concern because the audit system needs to provide reliable evidence of what happened and which operations were performed.

E1.5 New Regulations Require Twenty Year Record Retention

A requirement to keep records for twenty years would increase the amount of information that ERRS needs to store and manage. Incident reports, dispatch

details, intelligence records and audit logs may all need to remain available for the required period. Keeping these records for longer also means that personal and sensitive information remains exposed to potential risks for a longer time. The system would need to balance legal requirements with privacy and security responsibilities. Our prototype demonstrates basic retention decisions and legal hold scenarios, but it does not yet provide a complete long term archiving system or secure deletion process.

E2. Security Governance Decisions

The following decisions explain how I would handle each situation. The main aim is to protect the information stored in ERRS without creating unnecessary difficulties for emergency response teams.

E2.1 Research Organisation Access

I would not give researchers unrestricted access to intelligence records. They should only receive information that is approved for their research work. Personal details and RESTRICTED information should remain protected unless there is proper authorisation and a valid reason for access. This approach gives priority to confidentiality, even if it means researchers cannot access every record they request. The research view filtering already available in ERRS can support this decision. However, filtering does not guarantee complete anonymity because someone may still identify an individual by combining different pieces of information. This risk would need to be considered before sharing any research data.

E2.2 Emergency Responder Access

I would keep the existing restrictions on RESTRICTED intelligence rather than allowing every emergency responder to access it. If access is genuinely required, the request should be approved by someone who has the correct authority. A second authorised person should also review the request wherever the situation allows. The reason for access and the people involved should be recorded so that the activity can be checked later. This approach protects sensitive information and maintains accountability. However, it may take additional time during an emergency. The current prototype does not yet include this emergency approval process, so it would need to be developed and tested before the system could be used in a real environment.

E2.3 Regional Datacentre Failure

If a regional datacentre becomes permanently unavailable, I would prioritise keeping the data consistent rather than accepting updates that cannot be properly confirmed. If the required consensus majority is not available, the system should stop accepting protected commitments. Other services that do not depend on the unavailable consensus may continue where it is safe to do so. The affected services would need to be restored using independent infrastructure and a tested disaster recovery plan. This could mean that some operations remain unavailable for a period of time. However, accepting unconfirmed updates could create conflicting emergency records, which may cause more serious problems during incident management.

E2.4 Validator Credential Compromise

If validator credentials are stolen, I would revoke the affected credentials immediately and issue replacements. The system should check the updated key version before allowing the validator to take part in further operations. The audit records should also be reviewed to identify any suspicious activity that may have happened before the credentials were revoked. For a real deployment, I would recommend public key authentication and secure key storage instead of depending only on a shared HMAC secret. This decision gives priority to the integrity of the system and the reliability of its audit records. Validator operations may be interrupted while the credentials are replaced, but continuing to use potentially compromised credentials would create a greater risk.

E2.5 Twenty Year Record Retention

I would follow the twenty year retention requirement and ensure that stored records remain protected by suitable access restrictions. Records related to ongoing investigations or legal proceedings should be placed under a legal hold and should not be deleted while that hold is active. Retention periods and access permissions should also be reviewed regularly. This approach gives priority to legal compliance and the preservation of important records. However, it would increase storage costs and require regular management. The current prototype demonstrates basic retention decisions and legal holds, but a complete long term archiving and secure deletion process still needs to be developed.

E3. ERRS Security Assurance Statement

The ERRS prototype includes several security features that were developed and tested during the project. These include user authentication, role based access control, Bell LaPadula confidentiality, Biba integrity, information classification, audit verification and validator management. These features help control access to sensitive information and protect important system operations.

During testing, all 23 application tests passed. The project also includes nine consensus experiment records and five governance scenarios. These results show that the implemented features worked under the conditions we tested. However, passing the tests does not mean that the system is free from security weaknesses or protected against every possible attack.

There are still some limitations in the current implementation. All regions use the same SQLite database, while the consensus nodes run as Python objects in memory. The prototype does not have actual network replication between separate machines. Validator authentication also relies on a shared HMAC secret rather than a complete public key infrastructure. Staff accounts do not currently use multi factor authentication. Research data filtering cannot guarantee complete anonymity, and the retention feature does not yet support secure deletion of records.

The safe operation of ERRS would depend on several things being managed correctly. Information must be classified properly, user accounts must be trustworthy, credentials must remain protected and security policies must be configured correctly. These areas would need further testing and independent review before the system could be considered for real use.

Based on the current implementation and the evidence collected during testing, I would not recommend deploying the prototype as national emergency response infrastructure. At this stage, ERRS is more suitable as an educational project that demonstrates different security engineering principles. Before it could be considered for real world use, it would need separate regional infrastructure, permanent consensus storage, stronger authentication, secure cryptographic key management and a properly tested disaster recovery process. An independent security assessment would also be necessary before deployment.